Data Structures & Systems Design Assignment

Anay Shah, DJSCE Mumbai, B.Tech IT (Class of 2028)
Language: Python 3. Code and test output are included below.

Problem 1: LRU Cache

- I used a dictionary (key to node) and a doubly linked list. The dictionary finds a key in O(1). The list keeps the order, most recent at the front and least recent at the back.
- Because the dictionary gives me the node directly, I can remove it and move it to the front without searching. It is doubly linked so I can remove a node from the middle.
- get: return -1 if the key is missing, otherwise move the node to the front and return its value.
- put: if the key exists, remove the old node. If it is new and the cache is full, remove the node before the tail (least recently used) and delete its key from the dictionary. Then add the new node at the front.
- I used dummy head and tail nodes so I don't need special cases for empty or one item lists.
- Complexity: get and put are O(1) time. Space is O(capacity).
- Edge cases: updating an existing key, capacity 1, get on an empty cache.

lru_cache.py

```
class Node:
    def __init__(self, key, val):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head = Node(0, 0)   # dummy nodes so no edge cases
        self.tail = Node(0, 0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_front(self, node):
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self.remove(node)
        self.add_front(node)
        return node.val

    def put(self, key, value):
        if key in self.map:
            self.remove(self.map[key])
        elif len(self.map) == self.cap:
```

```
            lru = self.tail.prev
            self.remove(lru)
            del self.map[lru.key]
        node = Node(key, value)
        self.map[key] = node
        self.add_front(node)


# thread safe version: same thing but with a lock
import threading


class SafeLRUCache(LRUCache):
    def __init__(self, capacity):
        LRUCache.__init__(self, capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return LRUCache.get(self, key)

    def put(self, key, value):
        with self.lock:
            LRUCache.put(self, key, value)
```

Problem 2: Event Scheduler

- can_attend_all: sort by start time and check that each event starts at or after the previous one ends. Events that touch (end equals next start) are fine, so I use a strict less than.
- min_rooms_required: sort by start time and keep a min heap of end times, one for each room in use. If the smallest end time is less than or equal to the new start, that room is free, so I reuse it. Otherwise I add a new room. The heap size at the end is the answer.
- Example: [(0,30), (5,10), (15,20)] gives 2. The second event needs a new room, and the third reuses the room that freed up at 10.
- Complexity: O(n log n) time from the sort and heap operations. Space is O(n).
- Edge cases: unsorted input, touching events, identical events, nested events, empty list.

event_scheduler.py

```
import heapq


def can_attend_all(events):
    events = sorted(events)
    for i in range(1, len(events)):
        if events[i][0] < events[i - 1][1]:
            return False
    return True


def min_rooms_required(events):
    heap = []  # end times of rooms that are in use
    for start, end in sorted(events):
        if heap and heap[0] <= start:
            heapq.heappop(heap)  # reuse that room
        heapq.heappush(heap, end)
    return len(heap)


def assign_rooms(events):
    # returns a list of (event, room name) in the same order as the input
    order = sorted(range(len(events)), key=lambda i: events[i])
```

```
    busy = []   # (end time, room number)
    free = []   # room numbers that are free
    count = 0
    result = [None] * len(events)
    for i in order:
        start, end = events[i]
        while busy and busy[0][0] <= start:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            room = heapq.heappop(free)
        else:
            room = count
            count += 1
        heapq.heappush(busy, (end, room))
        result[i] = (events[i], "Room " + chr(65 + room))
    return result
```

Final Discussion

Function	Time	Space
LRUCache get / put	O(1)	O(capacity) overall
can_attend_all	O(n log n)	O(n)
min_rooms_required	O(n log n)	O(n)
assign_rooms	O(n log n)	O(n)

Why a hash map and a doubly linked list? The map gives O(1) lookup but has no order. The list gives order and O(1) moves, but searching it is O(n). Together both things are O(1). The cost is extra memory for the pointers.

How to assign room names? Keep (end time, room number) in the heap and a heap of free room numbers. For each event, free up the rooms that have ended, take the lowest free room or make a new one, and save the room against the event. This is assign_rooms in the code. It is still O(n log n).

How to make the LRU cache thread safe? Put a lock around get and put, as in SafeLRUCache. get also needs the lock because it changes the list order. For heavy traffic I would split the cache into a few smaller caches by hash(key) % N, each with its own lock.

Tests

tests.py

```
import threading
from lru_cache import LRUCache, SafeLRUCache
from event_scheduler import can_attend_all, min_rooms_required, assign_rooms


def test_lru_basic():
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    c.put(3, 3)            # evicts 2
    assert c.get(2) == -1
    c.put(4, 4)            # evicts 1
    assert c.get(1) == -1
    assert c.get(3) == 3
    assert c.get(4) == 4
```

```
def test_lru_update():
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    c.put(1, 10)           # 1 is now most recent
    c.put(3, 3)            # evicts 2
    assert c.get(1) == 10
    assert c.get(2) == -1


def test_lru_edge():
    c = LRUCache(1)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == -1
    assert c.get(2) == 2
    assert LRUCache(3).get(5) == -1


def test_threads():
    c = SafeLRUCache(100)

    def work(n):
        for i in range(1000):
            c.put((n, i % 150), i)
            c.get((n, i % 150))

    ts = [threading.Thread(target=work, args=(n,)) for n in range(8)]
    for t in ts:
        t.start()
    for t in ts:
        t.join()
    assert len(c.map) == 100


def test_can_attend():
    assert can_attend_all([(9, 10), (10, 11), (11, 12)]) == True
    assert can_attend_all([(0, 30), (5, 10), (15, 20)]) == False
    assert can_attend_all([]) == True


def test_min_rooms():
    assert min_rooms_required([(9, 10), (10, 11), (11, 12)]) == 1
    assert min_rooms_required([(0, 30), (5, 10), (15, 20)]) == 2
    assert min_rooms_required([(11, 12), (9, 10), (10, 11)]) == 1   # unsorted
    assert min_rooms_required([(1, 5)] * 4) == 4
    assert min_rooms_required([(1, 10), (2, 9), (3, 8)]) == 3
    assert min_rooms_required([]) == 0


def test_assign_rooms():
    got = assign_rooms([(0, 30), (5, 10), (10, 20)])
    assert got == [((0, 30), "Room A"), ((5, 10), "Room B"), ((10, 20), "Room B")]


tests = [test_lru_basic, test_lru_update, test_lru_edge, test_threads,
         test_can_attend, test_min_rooms, test_assign_rooms]
for t in tests:
    t()
    print("passed:", t.__name__)
print("all", len(tests), "tests passed")
```

Output (python tests.py)

```
passed: test_lru_basic
passed: test_lru_update
```

```
passed: test_lru_edge
passed: test_threads
passed: test_can_attend
passed: test_min_rooms
passed: test_assign_rooms
all 7 tests passed
```